

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto: PixelLang

PONDERACIÓN: 15 pts

 **Tiempo estimado: 10 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Resumen Ejecutivo	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema o necesidad a resolver	4
4.2 Alcance del proyecto	4
4.3 Recursos y herramientas a utilizar	4
4.4 Entregables	4
5. Material de apoyo	5
6. Metodología	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Detalle de la Calificación	7
8.3 Comentarios Generales	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad técnica	El estudiante deberá diseñar un analizador léxico correcto, reportar errores de forma clara y generar resultados reproducibles a partir del archivo de entrada.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

- Construir un analizador léxico manual para reconocer tokens de un lenguaje formal sencillo, gestionando el flujo de caracteres, lexemas, posiciones y errores léxicos.
- Interpretar una secuencia de tokens válidos para producir una salida visual en forma de tablero pixelado.
- Documentar y presentar reportes técnicos que evidencien el funcionamiento del analizador y el resultado generado.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Pensamiento analítico: descomponer un lenguaje en lexemas, tokens y reglas de reconocimiento.
- Atención al detalle: controlar filas, columnas, caracteres válidos y formato hexadecimal.
- Comunicación técnica: presentar reportes claros de tokens, errores e instrucciones procesadas.
- Creatividad aplicada: diseñar una salida visual a partir de un lenguaje formal.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Implementar un analizador léxico para PixelLang.	Reconocer tokens, reportar errores y generar un tablero pixelado funcional.	Mediante lectura carácter por carácter, tabla de tokens y validación del formato definido.	Para comprender la primera fase de un compilador y su aplicación en un lenguaje visual.	Durante el periodo definido por el tutor académico. Fecha de entrega: 07/06/2026 a las 12:00 hrs.

3. Resumen Ejecutivo

PixelLang es una práctica orientada a la implementación de un analizador léxico para un pequeño lenguaje diseñado para crear imágenes tipo pixel art. El estudiante deberá leer un flujo de caracteres, identificar lexemas, clasificar tokens, registrar la posición de cada token y reportar errores léxicos cuando aparezcan caracteres o formatos no válidos.

El lenguaje estará delimitado por un bloque principal PixelLang, un bloque de configuración config y un bloque de instrucciones paint. A partir de las instrucciones reconocidas, el programa deberá generar un tablero visual donde se pinten píxeles individuales, líneas horizontales, líneas verticales, áreas rectangulares y el tablero completo.

La prioridad de la práctica es el análisis léxico. La generación del tablero funcionará como evidencia visual del correcto reconocimiento e interpretación básica de la secuencia de tokens.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Se requiere desarrollar una aplicación que permita ingresar código escrito en PixelLang y analizarlo léxicamente. El programa deberá reconocer las palabras reservadas, símbolos, números, cadenas, colores hexadecimales y separadores definidos por el lenguaje. Luego del análisis, si no existen errores léxicos, el sistema deberá construir un tablero de píxeles según las instrucciones proporcionadas. La interfaz deberá mostrar el código fuente, ejecutar el análisis, visualizar el tablero generado y permitir consultar los reportes correspondientes

Lenguaje propuesto: PixelLang

La estructura general del archivo de entrada deberá respetar el siguiente formato:

```
PixelLang {  
  
  config {  
    BOARD: columns 10 rows 8;  
    NAME: "artepixel";  
  },  
  paint{  
    PIXEL: 3 4 #FF0000;  
    HORIZONTAL: 3 2 7 #80BCE8;  
    VERTICAL: 5 1 6 #FFD966;  
    FILLOUT: 2 2 6 6 #CCCCCC;  
    PAINT-ALL: #FFD966;  
  }  
}
```

Reglas generales del lenguaje

- El archivo debe iniciar con la palabra reservada PixelLang seguida de una llave de apertura { y finalizar con una llave de cierre }.
- El bloque config define la configuración del tablero y debe incluir BOARD y NAME.
- BOARD define el número de columnas y filas del tablero: BOARD: columns <numero> rows <numero>;
- NAME define el nombre de la imagen que será generada: NAME: "nombre";.
- El bloque paint contiene las instrucciones de pintado que serán aplicadas sobre el tablero.
- Las coordenadas se manejarán como (x, y), donde x representa la columna y y representa la fila. La coordenada (0,0) estará en la esquina superior izquierda.
- Los colores deben escribirse en formato hexadecimal: #RRGGBB. Ejemplo: #FF0000.
- Cuando dos instrucciones pinten la misma celda, prevalecerá el color de la última instrucción válida procesada.
- Los números deberán ser enteros positivos o cero. No se permitirán coordenadas negativas.
- El análisis léxico debe registrar fila y columna de cada token reconocido y de cada error encontrado.

Instrucciones de PixelLang

Instrucción	Sintaxis	Descripción
BOARD	BOARD: columns 10 rows 8;	Crea un tablero de 10 columnas y 8 filas.
NAME	NAME: "artepixel";	Define el nombre que tendrá la imagen al generarse. Ejemplo: artpixel.png
PIXEL	PIXEL: 3 4 #FF0000;	Pinta la celda ubicada en la columna 3, fila 4, con color rojo.
HORIZONTAL	HORIZONTAL: 3 2 7 #80BCE8;	Pinta una línea horizontal en la fila 3, desde la

		columna 2 hasta la columna 7.
VERTICAL	VERTICAL: 5 1 6 #FFD966;	Pinta una línea vertical en la columna 5, desde la fila 1 hasta la fila 6.
FILLOUT	FILLOUT: 2 2 6 6 #CCCCCC;	Llena un área rectangular desde la coordenada inicial (2,2) hasta la coordenada final (6,6).
PAINT-ALL	PAINT-ALL: #FFD966;	Pinta todas las celdas del tablero con el color indicado.

Las siguientes figuras muestran el efecto visual esperado de las instrucciones principales:

BOARD

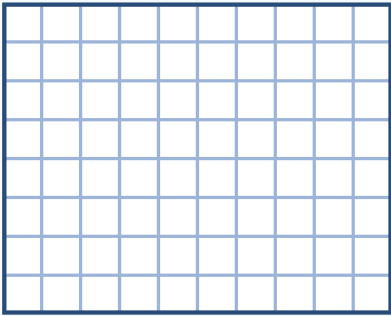
Define el tamaño del tablero: columnas y filas.

```
config {  
  BOARD: columns 10 rows 8;  
  NAME: "artepixel";  
}
```

filas (y)

0 1 2 3 4 5 6 7

0 1 2 3 4 5 6 7 8 9



columnas (x)

PIXEL

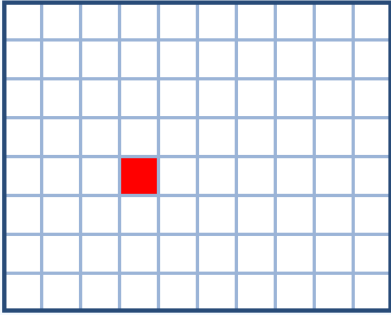
Pinta una única celda en la coordenada (x, y).

```
paint {  
  PIXEL: 3 4 #FF0000;  
}
```

filas (y)

0 1 2 3 4 5 6 7

0 1 2 3 4 5 6 7 8 9

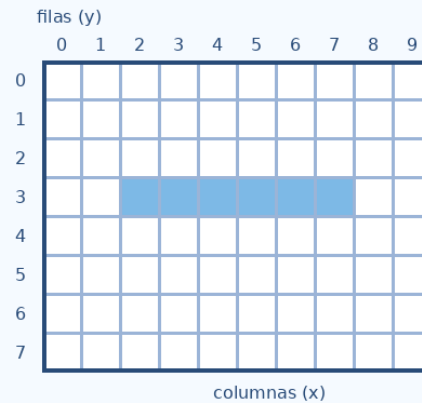


columnas (x)

HORIZONTAL

Pinta una fila desde xInicio hasta xFin.

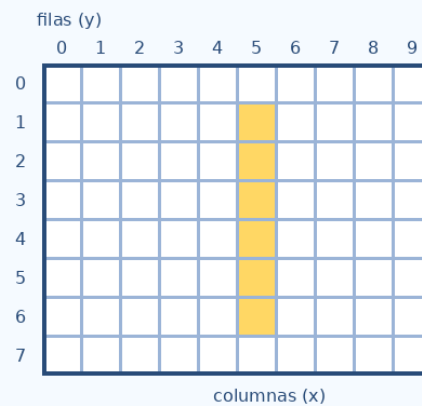
```
paint {  
  HORIZONTAL: 3 2 7 #80BCE8;  
}
```



VERTICAL

Pinta una columna desde yInicio hasta yFin.

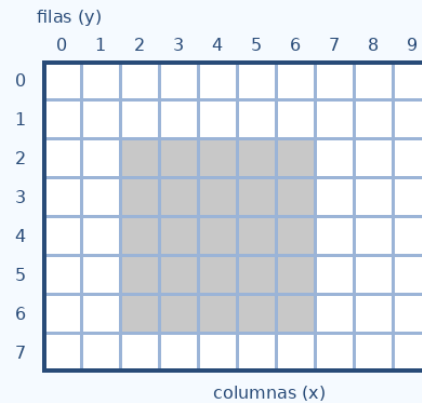
```
paint {  
  VERTICAL: 5 1 6 #FFD966;  
}
```



FILLOUT

Llena un área rectangular dentro del tablero.

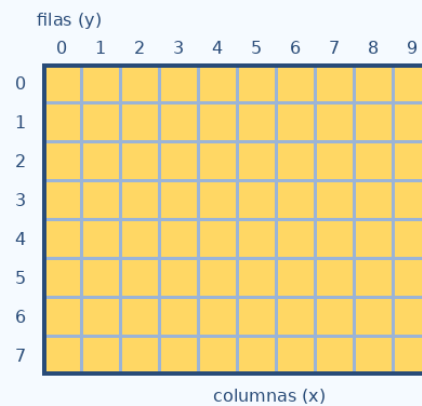
```
paint {  
  FILLOUT: 2 2 6 6 #CCCCC;  
}
```



PAINT-ALL

Pinta todas las celdas del tablero con un color.

```
paint {  
  PAINT-ALL: #FFD966;  
}
```



Reportes requeridos

Reporte	Contenido esperado
Reporte de tokens	Tabla con número, lexema, token, fila y columna. Este reporte es obligatorio y debe mostrarse desde la interfaz.
Reporte de errores léxicos	Listado de caracteres, símbolos o lexemas inválidos, con descripción, fila y columna.
Reporte de colores utilizados	Colores hexadecimales usados, cantidad de veces invocados y cantidad de celdas afectadas.
Reporte del tablero generado	Dimensiones, nombre de la imagen, total de celdas, celdas pintadas, celdas vacías y colores utilizados.

Reporte de coordenadas pintadas	Detalle de cada celda pintada indicando x, y, color e instrucción de origen.
---------------------------------	--

Interfaz mínima requerida

- Área de texto para ingresar o cargar el código fuente de PixellLang.
- Botón Analizar para ejecutar el análisis léxico y construir los reportes.
- Área de visualización del tablero pixelado generado.
- Botón Ver reporte de tokens.
- Botón Ver reporte de errores.
- Botón Ver reporte de tablero generado.
- Opción para generar o exportar la imagen usando el valor definido en NAME.
- Botón Limpiar para reiniciar la entrada, reportes y tablero.

4.2 Alcance del proyecto

Alcance obligatorio:

- Implementar un analizador léxico para PixellLang mediante lectura del flujo de caracteres.
- Reconocer todos los tokens mínimos establecidos en la tabla de tokens.
- Registrar fila y columna de cada token reconocido.
- Detectar y reportar errores léxicos.
- Implementar una interfaz con textarea, botón Analizar, visualización del tablero y botones para reportes.
- Interpretar las instrucciones BOARD, NAME, PIXEL, HORIZONTAL, VERTICAL, FILLOUT y PAINT-ALL para generar el tablero.
- Permitir exportar o generar la imagen con el nombre indicado en NAME.
- Generar los reportes obligatorios solicitados.

Alcance opcional:

- Permitir carga de archivos con extensión definida por el tutor, por ejemplo .pxl.
- Agregar vista previa de coordenadas al pasar el cursor sobre el tablero.
- Permitir descarga de reportes en HTML o PDF.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Software	Python para implementar el analizador léxico y la aplicación.

Obligatorio	Interfaz	Textarea, botones de análisis y área de visualización del tablero.
Obligatorio	Lógica	Lectura carácter por carácter, clasificación de lexemas y generación de tabla de tokens.
Obligatorio	Reportes	Reportes de tokens, errores y tablero generado.
Opcional	Exportación	Generación de imagen PNG o reporte HTML/PDF.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código fuente	Proyecto completo desarrollado en Python], con instrucciones de ejecución.
Reporte de tokens	Tabla con lexema, token, fila y columna.
Reporte de errores	Listado de errores léxicos detectados con posición y descripción.
Reportes complementarios	Colores utilizados, tablero generado y coordenadas pintadas.
Documentación	Manual breve de usuario.

5. Material de apoyo

Categoría (Manual /	Link	Descripción
---------------------	------	-------------

Documentación oficial/ Ejemplo / Etc)		
Ejemplos en clase		Ejemplos prácticos en clase

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Pasos a seguir:

1. Paso 1: Estudiar el lenguaje PixelLang. Identificar palabras reservadas, símbolos, números, cadenas, colores y estructura general del archivo.
2. Paso 2: Diseñar la tabla de tokens. Definir el nombre de cada token, el lexema que lo produce y su descripción.
3. Paso 3: Implementar el flujo de lectura. Leer el texto carácter por carácter, actualizando fila y columna.
4. Paso 4: Reconocer lexemas. Construir lexemas para palabras reservadas, números, cadenas y colores hexadecimales.
5. Paso 5: Registrar errores léxicos. Detectar caracteres no permitidos, cadenas no cerradas y colores con formato inválido.
6. Paso 6: Generar reportes. Construir tablas de tokens, errores, instrucciones reconocidas, colores y tablero.
7. Paso 7: Interpretar instrucciones válidas. Crear el tablero con BOARD, usar NAME para la imagen y aplicar instrucciones paint en orden.
8. Paso 8: Diseñar la interfaz. Agregar textarea, botón Analizar, visualización del tablero y botones para reportes.
9. Paso 9: Probar con varios archivos. Validar entradas correctas, errores léxicos y casos límite como coordenadas fuera del tablero.

Recomendaciones

- Separar la lógica del analizador léxico de la lógica de interfaz.
- Guardar tokens y errores en estructuras de datos antes de mostrarlos.
- Probar primero con entradas pequeñas antes de generar imágenes complejas.
- Documentar claramente la interpretación de coordenadas y el origen del tablero.
- Validar que el color hexadecimal tenga exactamente seis dígitos válidos después del símbolo #.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Publicación de la práctica	02/06/2026	07/06/2026 12:00 hrs

Entrega final		07/06/2026 12:00 hrs
---------------	--	----------------------

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la practica, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
Entrega en fecha	La práctica debe entregarse en la fecha establecida por el tutor académico: 07/06/2026 12:00 hrs.	
Ejecución del proyecto	El proyecto debe poder ejecutarse sin errores críticos en el entorno indicado.	
Analizador propio	El analizador léxico debe ser implementado por el estudiante. No se aceptan librerías que resuelvan el análisis completo del lenguaje.	
Reportes obligatorios	Debe incluir, como mínimo, reporte de tokens y reporte de errores léxicos.	
Código fuente	Debe entregar el código fuente completo y organizado.	

8.2 Detalle de la Calificación

	Criterio/Nivel		Nivel de evaluación		
No.	Criterio de evaluación		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	Interfaz clara, reportes accesibles, visualización correcta del tablero e imagen generada con NAME.	Interfaz incompleta, reportes difíciles de consultar o tablero poco claro.		1
1.1 1.2	Diseño de interfaz Visualización y exportación	Incluye textarea, botón Analizar, visualización del tablero y botones de reportes.	Faltan componentes esenciales o la interacción no es clara.		1.1 1.2
		El tablero se genera correctamente	La imagen no se genera, no respeta NAME o la visualización es incorrecta.		

Lenguajes Formales y de Programación
Practica única - vigente para las vacaciones de junio de 2026

		e y se puede exportar o generar imagen con el nombre definido.			
1.3 2	Presentación y documentación Conocimientos	Incluye instrucciones de uso, ejemplos y explicación clara del flujo léxico.	Documentación incompleta o poco comprensible.		1.3 2
		Implementa adecuadamente el análisis léxico, tokens, errores e interpretación básica de instrucciones.	Presenta fallas importantes en el reconocimiento de tokens o generación del tablero.		
2.1	Reconocimiento de tokens	Reconoce palabras reservadas, símbolos, números, cadenas y	Omite tokens importantes o no registra posición correctamente.		2.1

		colores hexadecimal es con fila y columna.			
2.2	Manejo de errores léxicos	Detecta caracteres no válidos, colores incorrectos y cadenas no cerradas con posición.	No detecta errores o los reporta de manera ambigua.		2.2
2.3 2.4	Interpretación de instrucciones Reportes técnicos	Aplica correctamente BOARD, NAME, PIXEL, HORIZONTAL, VERTICAL, FILLOUT y PAINT-ALL.	Las instrucciones se interpretan de forma parcial o incorrecta.		2.3 2.4
		Genera reportes de tokens, errores, instrucciones	Faltan reportes o la información generada es incompleta.		

		s, colores, tablero y coordenadas pintadas.			
1	Total Habilidades				1
		Interfaz clara, reportes accesibles, visualización correcta del tablero e imagen generada con NAME.	Interfaz incompleta, reportes difíciles de consultar o tablero poco claro.		
1.1	Diseño de interfaz	Incluye textarea, botón Analizar, visualización del tablero y botones de reportes.	Faltan componentes esenciales o la interacción no es clara.		1.1
1.2	Visualización y exportación	El tablero se genera	La imagen no se genera, no respeta NAME o la visualización es incorrecta.		1.2

		correctamente y se puede exportar o generar imagen con el nombre definido.			
--	--	--	--	--	--

8.3 Comentarios Generales

Deberá agregar al auxiliar a su repositorio en github, el nombre del repositorio deberá llevar el siguiente formato **LFPVJ_#Carnet**, y dentro del repositorio deberá crear una carpeta llamada Practica_Unica, donde deberá subir todo lo relacionado con su práctica. El usuario del auxiliar es el siguiente **JohnAlds**.